Task 10: Ideal High-Pass Filter (IHPF) in Frequency Domain

Blocks low-frequency elements within radial distance D_0 to highlight subtle edges and sharp image transitions.

This program performs Ideal High-Pass Filtering (IHPF) using 2D Fourier Transform. It is a Frequency Domain Filtering technique in Digital Image Processing.

The main purpose of this program is to remove Low-Frequency Components and preserve High-Frequency Components of an image.

I will explain the concept, complete code, each step, and possible viva questions so you can explain the program in your exam.

1. What is Ideal High-Pass Filtering?

Ideal High-Pass Filtering is an image processing technique that allows High-Frequency Components to pass and blocks Low-Frequency Components.

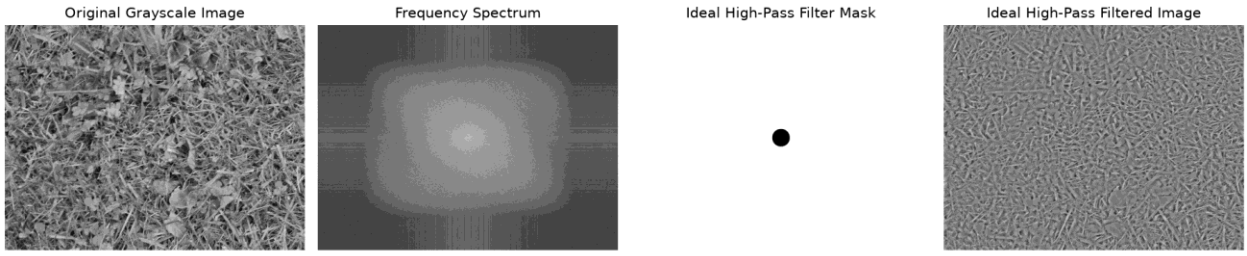
An image contains two types of frequency:

1. Low Frequency:
   - Smooth areas of an image.
   - Background and gradual intensity changes.
   - Large structures and uniform regions.
2. High Frequency:
   - Edges and boundaries.
   - Fine details and textures.
   - Rapid changes in pixel intensity.

Main concept:

- Low Frequency = Blocked (0)
- High Frequency = Passed (1)

Difference between Low-Pass and High-Pass Filtering



Keeps edges and fine details while removing smooth components.

In your uploaded result:

- Original Grayscale Image: Original grass image.

- Frequency Spectrum: Shows the frequency distribution.
- Ideal High-Pass Filter Mask: Black center and white outside.
- Ideal High-Pass Filtered Image: Shows the remaining high-frequency details.

2. Complete Corrected Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

image_path = "PNG/Task_10_Ideal_High_Pass_Filter.png"

img = Image.open(image_path).convert("RGB")
img_array = np.asarray(img, dtype=np.float32)

gray = (
    0.299 * img_array[:, :, 0] +
    0.587 * img_array[:, :, 1] +
    0.114 * img_array[:, :, 2]
)

F = np.fft.fft2(gray)
F_shifted = np.fft.fftshift(F)

rows, cols = gray.shape
center_row = rows // 2
center_col = cols // 2
D0 = 30
mask = np.zeros((rows, cols), dtype=np.float32)

for i in range(rows):
    for j in range(cols):
        distance = np.sqrt(
            (i - center_row) ** 2 +
            (j - center_col) ** 2
        )
        if distance > D0:
```

```
            mask[i, j] = 1

G = F_shifted * mask
G_ishift = np.fft.ifftshift(G)
filtered_image = np.fft.ifft2(G_ishift)
filtered_image = np.real(filtered_image)
min_val = np.min(filtered_image)
max_val = np.max(filtered_image)

if max_val > min_val:
    output = (filtered_image - min_val) / (max_val - min_val)
    output = output * 255
else:
    output = np.zeros_like(filtered_image)

plt.figure(figsize=(15, 5))

plt.subplot(1, 4, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(np.log(1 + np.abs(F_shifted)), cmap="gray")
plt.title("Frequency Spectrum")
plt.axis("off")

plt.subplot(1, 4, 3)
plt.imshow(mask, cmap="gray")
plt.title("Ideal High-Pass Filter Mask")
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(output, cmap="gray")
plt.title("Ideal High-Pass Filtered Image")
```

```
plt.axis("off")

plt.tight_layout()
plt.show()
```

3. Code Explanation (Step by Step)

Step 1: Import Libraries

import numpy as npimport matplotlib.pyplot as pltfrom PIL import Image

এখানে তিনটি Library Import করা হয়েছে।

- numpy: Mathematical Operations এবং Fourier Transform করার জন্য।
- matplotlib.pyplot: Image Display করার জন্য।
- PIL: Image Open এবং Convert করার জন্য।

os Library ব্যবহার করা হয়েছে Output Folder তৈরি করার জন্য।

Step 2: Set Image Path

image_path = "PNG/Task_10_Ideal_High_Pass_Filter.png"

এখানে Original Image-এর Location নির্ধারণ করা হয়েছে।

এই Location থেকে আমাদের Input Image Load হবে।

Step 3: Load RGB Image

img = Image.open(image_path).convert("RGB")img_array = np.asarray(img, dtype=np.float32)

প্রথম লাইনে Image Open করে RGB Format-এ Convert করা হয়েছে।

দ্বিতীয় লাইনে Image-কে NumPy Array-তে Convert করা হয়েছে।

এখানে float32 ব্যবহার করা হয়েছে Mathematical Calculation সহজ করার জন্য।

Step 4: Convert RGB to Grayscale

gray = (0.299 * img_array[:, :, 0] + 0.587 * img_array[:, :, 1] + 0.114 * img_array[:, :, 2])

এখানে RGB Image-কে Grayscale Image-এ Convert করা হয়েছে।

RGB Image-এ তিনটি Channel থাকে:

- Red = 0.299
- Green = 0.587
- Blue = 0.114

এই তিনটি Channel-এর Weighted Sum ব্যবহার করে Grayscale Image তৈরি করা হয়েছে।

সহজভাবে, Color Image-কে Black and White Image-এ Convert করা হচ্ছে।

Step 5: Apply 2D Fourier Transform

F = np.fft.fft2(gray)

এটি আমাদের প্রোগ্রামের গুরুত্বপূর্ণ অংশ।

এখানে np.fft.fft2() ব্যবহার করে Grayscale Image-কে Spatial Domain থেকে Frequency Domain-এ Convert করা হয়েছে।

এখন Image-এর বিভিন্ন Frequency Component পাওয়া যাবে।

Step 6: Shift Zero Frequency to Center

F_shifted = np.fft.fftshift(F)

Fourier Transform করার পর Zero Frequency সাধারণত Spectrum-এর Corner-এ থাকে।

এখানে fftshift() ব্যবহার করে Zero Frequency-কে Image-এর Center-এ নিয়ে আসা হয়েছে।

এতে Center থেকে Distance Calculate করে Filter তৈরি করা সহজ হয়।

মনে রাখবে:

- Center = Low Frequency
- Outside = High Frequency

Step 7: Get Image Dimensions

rows, cols = gray.shapecenter_row = rows // 2center_col = cols // 2

এখানে Image-এর Row এবং Column সংখ্যা বের করা হয়েছে।

- rows = Image Height।
- cols = Image Width।
- center_row = Center Row।
- center_col = Center Column।

// ব্যবহার করে Integer Division করা হয়েছে।

Step 8: Set Cutoff Frequency

D0 = 30

এখানে Cutoff Frequency নির্ধারণ করা হয়েছে।

D0 নির্ধারণ করে Center-এর কত দূরত্বের Frequency Block হবে।

- D0 ছোট হলে কম Low Frequency Block হবে।
- D0 বড় হলে বেশি Low Frequency Block হবে।

অর্থাৎ D0 বাড়ালে High-Pass Filter-এর মধ্যে আরও বেশি Frequency বাদ পড়ে এবং তুলনামূলক কম Frequency অবশিষ্ট থাকে।

Step 9: Create High-Pass Filter Mask

mask = np.zeros((rows, cols), dtype=np.float32)

এখানে একটি Blank Filter Mask তৈরি করা হয়েছে।

np.zeros() ব্যবহার করার কারণে শুরুতে Mask-এর প্রতিটি Pixel Value হবে 0।

অর্থাৎ শুরুতে সব Frequency Block থাকবে।

এরপর আমরা High Frequency-গুলোকে 1 করে দেব।

Step 10: Calculate Distance

for i in range(rows): for j in range(cols): distance = np.sqrt((i - center_row) ** 2 + (j - center_col) ** 2)

এখানে Nested Loop ব্যবহার করে প্রতিটি Frequency Position থেকে Center-এর Distance বের করা হয়েছে।

এখানে Euclidean Distance Formula ব্যবহার করা হয়েছে।

$$ D(i,j)=\sqrt{(i-c_r)^2+(j-c_c)^2} $$

যেখানে:

- i = Current Row Position।
- j = Current Column Position।
- center_row = Center Row।
- center_col = Center Column।
- distance = Center থেকে Current Position-এর দূরত্ব।

Step 11: Apply High-Pass Filter Condition

if distance > D0: mask[i, j] = 1

এটি High-Pass Filtering-এর সবচেয়ে গুরুত্বপূর্ণ Logic।

এখানে Condition Check করা হয়েছে।

যদি Distance Cutoff Frequency-এর চেয়ে বেশি হয়, তাহলে Mask-এর Value 1 হবে।

অন্যথায় Value 0 থাকবে।

অর্থাৎ:

- 1 = High Frequency Pass করবে।
- 0 = Low Frequency Block হবে।

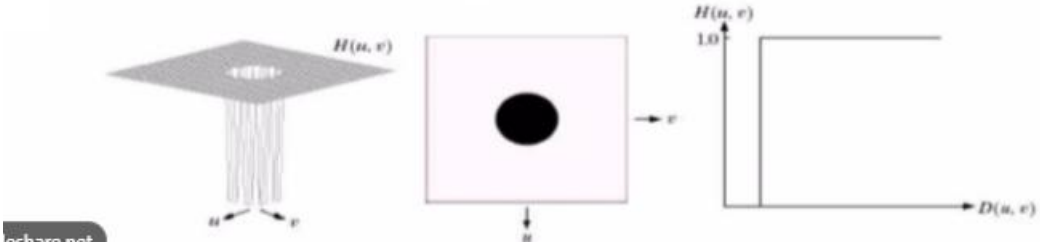
Ideal High-Pass Filter Mask

IDEAL HIGH PASS FILTER

The ideal high pass filter is given as:

$$H(u,v) = \begin{cases} 0 & \text{if } D(u,v) \le D_0 \\ 1 & \text{if } D(u,v) > D_0 \end{cases}$$

where D_0 is the cut off distance as before



White = 1 (High Frequency Pass)

Black = 0 (Low Frequency Block)

তোমার Output-এ Mask-এর Center Black এবং বাইরের অংশ White দেখা যাচ্ছে।

কারণ Center-এর Low Frequency Block করা হয়েছে এবং বাইরের High Frequency Pass করা হয়েছে।

Step 12: Apply High-Pass Filter

G = F_shifted * mask

এখানে Fourier Spectrum-এর সঙ্গে Filter Mask Multiply করা হয়েছে।

- Mask-এর Value 1 হলে Frequency অপরিবর্তিত থাকবে।
- Mask-এর Value 0 হলে Frequency শূন্য হয়ে যাবে।

অর্থাৎ Low Frequency বাদ দিয়ে High Frequency রাখা হচ্ছে।

এটাই আমাদের Filtering Operation।

Step 13: Apply Inverse Fourier Transform

G_ishift = np.fft.ifftshift(G)filtered_image = np.fft.ifft2(G_ishift)

এখানে Frequency Domain থেকে আবার Spatial Domain-এ ফিরে আসা হয়েছে।

প্রথম লাইনে ifftshift() দিয়ে Frequency Arrangement আগের অবস্থায় ফিরিয়ে নেওয়া হয়েছে।

দ্বিতীয় লাইনে ifft2() ব্যবহার করে Filtered Frequency Data থেকে নতুন Image তৈরি করা হয়েছে।

সহজভাবে:

- fft2() = Image থেকে Frequency।

- ifft2() = Frequency থেকে Image।

Step 14: Extract Real Values

filtered_image = np.real(filtered_image)

Inverse Fourier Transform-এর Result Complex Number হতে পারে।

এখানে np.real() ব্যবহার করে শুধু Real Part নেওয়া হয়েছে।

এর ফলে আমরা Image-এর জন্য প্রয়োজনীয় Real Pixel Value পাই।

Step 15: Normalize Pixel Values

min_val = np.min(filtered_image)max_val = np.max(filtered_image)

এখানে Filtered Image-এর Minimum এবং Maximum Pixel Value বের করা হয়েছে।

এরপর:

if max_val > min_val: output = (filtered_image - min_val) / (max_val - min_val) output = output * 255else: output = np.zeros_like(filtered_image)

এখানে Normalization করা হয়েছে।

Normalization-এর মাধ্যমে Pixel Value-গুলোকে 0 থেকে 255 Range-এর মধ্যে নিয়ে আসা হয়েছে।

এতে Filtered Image সহজে Display করা যায়।

Step 16: Display Image

plt.figure(figsize=(15, 5))

এখানে Display Window-এর Size নির্ধারণ করা হয়েছে।

এরপর:

plt.subplot(1, 4, 1)

ব্যবহার করে ১টি Row এবং ৪টি Column-এর Layout তৈরি করা হয়েছে।

এখানে চারটি Image পাশাপাশি Display করা হবে:

1. Original Grayscale Image
2. Frequency Spectrum
3. Ideal High-Pass Filter Mask
4. Ideal High-Pass Filtered Image

সবশেষে:

plt.tight_layout()plt.show()

plt.tight_layout() Image-গুলোর মধ্যে Spacing ঠিক করে এবং plt.show() সব Image Display করে।

বাংলায় ব্যাখ্যা:

প্রথমে আমি প্রয়োজনীয় Library Import করেছি এবং Original RGB Image-টি Grayscale-এ Convert করেছি। এরপর Image-টিকে NumPy Array-তে Convert করেছি।
তারপর 2D Fourier Transform ব্যবহার করে Image-টিকে Spatial Domain থেকে Frequency Domain-এ Convert করেছি। এরপর FFT Shift ব্যবহার করে Zero Frequency Component-কে Center-এ নিয়ে এসেছি।
পরবর্তীতে আমি 30 Cutoff Frequency নির্ধারণ করে একটি Ideal High-Pass Filter Mask তৈরি করেছি। এখানে Center থেকে Distance 30-এর বেশি হলে Mask-এর Value 1 এবং বাকি Position-এ 0 রাখা হয়েছে।
এরপর Fourier Spectrum-এর সঙ্গে Filter Mask Multiply করেছি। ফলে Low Frequency বাদ পড়েছে এবং High Frequency বজায় রয়েছে।
সবশেষে Inverse Fourier Transform ব্যবহার করে Filtered Image তৈরি করেছি। এরপর Pixel Value Normalize করে Original Image, Frequency Spectrum, Filter Mask এবং Final Output পাশাপাশি Display করেছি।

–**5. Important Viva Questions and Answers**

Q1. What is Ideal High-Pass Filtering?

Answer: It is a frequency domain technique that allows high frequencies to pass and blocks low frequencies.

বাংলা: এটি এমন একটি Filtering Technique, যেখানে High Frequency রাখা হয় এবং Low Frequency বাদ দেওয়া হয়।

Q2. What is the difference between ILPF and IHPF?

Answer:

- ILPF passes Low Frequencies and blocks High Frequencies.
- IHPF passes High Frequencies and blocks Low Frequencies.

Q3. Why do we use Fourier Transform?

Answer: To convert an image from Spatial Domain to Frequency Domain.

Q4. What is the purpose of fftshift()?

Answer: It moves the zero-frequency component to the center of the spectrum.

Q5. What is Cutoff Frequency (D0)?

Answer: It determines the boundary between blocked and passed frequencies.

Q6. Why is the center of the High-Pass Mask black?

Answer: Because low-frequency components are blocked by setting their mask values to zero.

Q7. Why is the outside area of the mask white?

Answer: Because high-frequency components are allowed to pass by setting their mask values to one.

Q8. What does np.fft.ifft2() do?

Answer: It converts filtered frequency data back into a spatial domain image.

Q9. Why do we use Normalization?

Answer: To scale the filtered pixel values into the 0–255 range for image display.

Q10. What happens when D0 increases?

Answer: More low-frequency components are blocked, and fewer frequencies pass through the filter.

6. Complete Working Process



Key Caveats:

1. High-Pass Filtering preserves edges and fine details but can also preserve or amplify noise.
2. Increasing D0 blocks more low-frequency components.
3. Ideal High-Pass Filtering may produce ringing effects because of its abrupt frequency cutoff.
4. The normalization step improves visualization but does not change which frequencies were filtered.

Most important five lines for your exam:

F = np.fft.fft2(gray)F_shifted = np.fft.fftshift(F)if distance > D0: mask[i, j] = 1G = F_shifted * maskfiltered_image = np.fft.ifft2(np.fft.ifftshift(G))

Remember this sequence:

Grayscale → Fourier Transform → FFT Shift → High-Pass Mask → Filtering → Inverse Fourier Transform.

This is the core concept of your entire program.